Assignment 5

2026-10-16

Large language models (LLMs) are all the rage in AI. In this lab you will write software for working with *small* language models. The program should read a text file, infer a language model from it, and then use that model to generate new "text" (not necessarily correct), thus implementing an *Artificial Idiot*.

You are supposed to make this a full C++ project, organised so that it is easy for others to reuse.

- Use git and GitHub, Bitbucket, or GitLab for easy code sharing, distribution, and project management.
- Write a Makefile for easy compilation.
- Use exceptions to handle user errors.
- Write a small report, to be found in your git repository.

The computational problem

We want to construct a language model that operates on k-grams, or "words", of fixed length . The k-grams are created over an alphabet defined by the input, and may contain any character, so the k-grams are not restricted to linguistic words. The hope is that "real" words are generated by chance often enough to make the text look decent. A larger k means stronger constraints on the output and a higher probability of generating proper words.

The model is trained on an input text to extract two key components:

1. the frequency of each k-gram, and
2. the probability distribution over the *next* character following each such k-gram.

For every k-gram w, we examine the set of characters c that appear immediately after w in the text. This allows us to infer a conditional probability distribution over characters, denoted by $P(c|w)$, which represents the probability of observing character $c$given the preceding k-gram .

Let $n(w, c)$ be the number of times character c follows k-gram w in the input text, and let $n(w)$ be the number of occurrences of w that are followed by some character, i.e., $n(c) = \sum_c n(w, c)$. We then estimate the conditional probability as

$$P(c|w) = \frac{n(w, c)}{n(w)}$$

Note that an occurrence of w at the very end of the text is followed by nothing, and is therefore not counted in $n(w)$.

In the tables below, a space character is shown as $\sqcup$.

Example 1

If $k = 2$ and the input text is "ababac", then the bigram (2-gram) frequencies are:

bigram	f
`ab`	0.4
`ba`	0.4
`ac`	0.2

This is because there are five bigrams in the input text, and "ab" and "ba" appear twice each, while "ac" appears once.

Here are the character transition probabilities:

bigram	next char	$P(c\|w)$
`ab`	`a`	1.0
`ba`	`b`	0.5
`ba`	`c`	0.5

After "ab" in the input, only "a" has followed, whereas the bigram "ba" has been followed by either "b" or "c". The bigram "ac" only occurs at the end of the text, so it has no transitions at all.

Example 2

If $k = 3$ and the input text is "Hubba bubba", then the trigram frequencies are:

trigram	f
`Hub`	0.111
`ubb`	0.222
`bba`	0.222
`ba␣`	0.111
`a␣b`	0.111
`␣bu`	0.111
`bub`	0.111

Note that "ubb" and "bba" are found twice in the input string, and there are nine trigrams in the input text. Three trigrams contain a space, because that was part of the input.

The transition probabilities are a bit boring, since there is so little variation in the small input text:

trigram	next char	$P(c\|w)$
`Hub`	`b`	1.0
`ubb`	`a`	1.0
`bba`	`␣`	1.0
`ba␣`	`b`	1.0
`a␣b`	`u`	1.0
`␣bu`	`b`	1.0
`bub`	`b`	1.0

Note that "bba" occurs twice, but the second occurrence is at the end of the text, so only the first one contributes to $n(\mathbf{bba}) = 1$.

Pseudocounts

With the estimate above, a character that has never followed w in the training text gets probability zero, and a k-gram that is never followed by anything has no distribution at all. A common remedy is to add a *pseudocount* $\alpha \geq 0$ to every count. Let Σ be the alphabet, i.e., the set of distinct characters in the training text, and let $|\Sigma|$ be its size. The estimate then becomes

$$P(c|w) = \frac{n(w,c) + \alpha}{n(w) + \alpha|\Sigma|} \text{ for every } c \in \Sigma.$$

The probabilities still sum to one, and with $\alpha \geq 0$ every character in the alphabet has a positive probability after every \$k\$-gram, including \$k\$-grams that never occurred in the training text, for which $n(w) = 0$ and the distribution is uniform. With $\alpha = 0$ we get the original estimate back. A small α adds a little "creativity" to the output, while a large α drowns the information in the training text and gives random characters. (The case $\alpha = 1$ is known as *Laplace smoothing*.)

For example, with $\alpha = 1$ and the input text "ababac" from Example 1, the alphabet is $\Sigma = \{\mathbf{a},\mathbf{b},\mathbf{c}\}$, and the transition probabilities become:

bigram	`a`	`b`	`c`
`ab`	0.6	0.2	0.2
`ba`	0.2	0.4	0.4
`ac`	0.333	0.333	0.333

For instance, $P(\mathbf{a}|\mathbf{ab}) = (2 + 1)/(2 + 1 \cdot 3) = 0.6$.

Generating text

Once a model has been implemented, it can be used to generate new text. The basic algorithm is:

1. Choose a starting k-gram $w = w_1 w_2 \ldots w_k$ according to the k-gram frequencies.
2. Output w.
3. Look up the character distribution for w, and sample the next character c. If $\alpha = 0$ and w has no transitions (it only occurs at the end of the training text), stop.
4. Output c.
5. Set $w = w_2 \ldots w_k c$, i.e., remove the first character and append the sampled character c.
6. Go to 3, unless you have output the requested number of characters.

The requested number of characters includes the starting k-gram. If the generator gets stuck in step 3, it simply gives up, so the output may be shorter than requested. With $\alpha > 0$ this cannot happen, but note that w may then be a k-gram that never occurred in the training text, and your code must handle that.

The C++ standard library header `<random>` is helpful for generating random numbers, and for sampling discrete objects (like characters) from a probability distribution. Have a look at `std::discrete_distribution`.

The software

The goal is a Unix tool `slm` (for Small Language Model) that takes three arguments:

- An integer k that decides the word size.
- The name of a text file, containing any kind of text, that is used for training the model.
- An integer that decides how many characters to output, including the starting k-gram. It must be at least k.

In addition, the user can give the pseudocount with the option $-a$, placed before the three arguments. If the option is not given, $\alpha = 0$.

You should be able to use the model like this:

```
$ slm 2 moby_dick.txt 5
it we
```

Here "it we" is conceivable output: five characters from the model, starting with the bigram "it". With a pseudocount:

```
$ slm -a 0.01 3 moby_dick.txt 40
```

Training texts

Training texts in English are available on the course web. Use these texts. They are plain ASCII, so you may assume that each character in the text is one `char`. (Letters such as å, ä, and ö take up more than one `char` each in modern text files, which would complicate the lab.)

Handling user errors

Your program must handle user errors gracefully, using exceptions. User errors include at least:

- the wrong number of arguments,
- a k or an output length that is not a positive integer, or an output length smaller than k,
- an option `-a` that is not followed by a non-negative number, or an unknown option,
- a training file that cannot be opened, and
- a training text that is too short to contain any k-gram followed by a character.

Detect each error where it naturally occurs, for example in the code that reads the training file, and `throw` a suitable standard exception, such as `std::invalid_argument` or `std::runtime_error`, with a descriptive message. Catch the exceptions in `main`, print the message to `std::cerr`, and exit with a non-zero exit status. For example:

```
$ slm 2 no_such_file.txt 100
slm: cannot open file 'no_such_file.txt'
```

The code

You should use at least two classes for this project, and each class should be found in its own .cpp/.h files. The main function is to be found in a separate file, possibly with supporting code.

A Makefile should be used to compile the code. The Makefile should be placed in the project directory, i.e., the top directory of your repository, while the code is in `src`. When we download your project, we should be able to run `make` in the project directory to compile the program into the executable `slm`. The Makefile should also have a target `clean` that removes all generated files.

Hints

It will probably be helpful to use `std::map`, which is like a dictionary in Python.

The git repository

You are expected to use git for source code version control and management. We will look at your git commit history to assess whether you have made frequent commits. Your repository should contain:

- A file README.md or README.txt that briefly describes your software, how to compile the code, and how to use it. This is the file that GitHub, Bitbucket, and GitLab use to present the project. It is also, in effect, the lab report.
- The code, in a folder named `src`.
- The Makefile, in the top directory.
- A `.gitignore` file, so that generated files (object files and the executable) are not committed.

Submission

When you are ready, submit a link to your repository on the course web. Make sure that the teachers can access the repository: either make it public, or share it with the teachers. I am "arvestad" on both GitHub and GitLab, and "lars.arvestad@gmail.com" on BitBucket.

Grading

- **Correctness** (2p): Does `slm` build the model and generate text as specified, including output of the starting k-gram, the pseudocount option, and stopping when the generator gets stuck?
- **Error handling** (1p): Are user errors detected and reported using exceptions, as specified?
- **Code structure** (1p): Are there at least two classes, each in its own .cpp/.h files, and is `main` in a separate file?
- **Makefile** (2p): Does `make` in the project directory build `slm`, recompiling only what is necessary, and does `make clean` work?
- **Use of git** (2p): Has git been used actively and sensibly, with frequent commits, informative commit messages, and no generated files in the repository?
- **README** (1p): Does the README describe the software, how to compile it, and how to use it?

The sum of points from these criteria is then divided by 3 to get the final score for the assignment.